

Bank Churners

Tala Haddadin
School Of Applied Technical Sciences
German Jordanian University
Amman, Jordan
t.haddadin1@gju.edu.jo

Celine Alarmouti
School Of Applied Technical Sciences
German Jordanian University
Amman, Jordan
c.alarmouti@gju.edu.jo

Abstract— Machine learning (ML) is a sort of artificial intelligence (AI) that allows software programs to improve their prediction accuracy without being expressly designed to do so. In order to predict new output values, machine learning algorithms use past data as input. The primary goal is for computers to learn on their own, without the need for human involvement, and to change their behavior accordingly.

Keywords— Machine learning, Bank Churn, Attrited Customers and Existing Customers

I. INTRODUCTION

Customer retention is important in a variety of businesses as acquiring new customers is often more costly than keeping the current ones. Therefore, churn prediction has attracted great attention from both the business and academic worlds. We found a dataset for this topic by a bank manager that is facing an issue that more and more consumers are canceling their credit card services.

So, we are going to make a ML project that can predict who would cancel their credit card so that the bank would go out of their way and deliver decent support and influence consumers' choices in the opposite direction.

The main purpose of this project is to maintain the number of customers by making a useful pattern of their behavior based on the dataset we have; we will try to take the process on its simplest scale by classifying the Dataset that contains 10.000 customers' information and make conclusions from it so the model will warn us before any customer may cancel their card.

We will use around 75% for learning purposes, the other 25% we will try to find the accuracy of our model.

II. DATA DESCRIPTION

A. Data

We obtained a dataset from Kaggle, it contains around 10,000 customers, and 23 features.

- **Attrition_Flag**: Internal event (customer activity) variable - if the account is closed then 1(Attrited Customer) else 0(Existing Customer)
- **Customer_Age**: Demographic variable - Customer's Age in Years
- **Gender**: Demographic variable - M=Male, F=Female
- **Dependent_count**: Demographic variable - Number of dependents
- **Education_Level**: Demographic variable - Educational Qualification of the account holder (example: high school, college graduate, etc.)
- **Marital_Status**: Demographic variable - Married, Single, Divorced, Unknown

- **Income_Category**: Demographic variable - Annual Income Category of the account holder (< 40K, 40K - 60K, 60K- 80K, 80K- 120K,> \$ 120K, Unknown)
- **Card_Category**: Product Variable - Type of Card (Blue, Silver, Gold, Platinum)
- **Months_on_book**: Period of relationship with bank
- **Total_Relationship_Count**: Total no. of products held by the customer
- **Months_Inactive_12_mon**: No. of months inactive in the last 12 months
- **Contacts_Count_12_mon**: No. of Contacts in the last 12 months
- **Credit_Limit**: Credit Limit on the Credit Card
- **Total_Revolving_Bal**: Total Revolving Balance on the Credit Card
- **Avg_Open_To_Buy**: Open to Buy Credit Line (Average of last 12 months)
- **Total_Amt_Chng_Q4_Q1**: Change in Transaction Amount (Q4 over Q1)
- **Total_Trans_Amt**: Total Transaction Amount (Last 12 months)
- **Total_Trans_Ct**: Total Transaction Count (Last 12 months)
- **Total_Ct_Chng_Q4_Q1**: Change in Transaction Count (Q4 over Q1)
- **Avg_Utilization_Ratio**: Average Card Utilization Ratio

B. Reading the Data

We imported pandas library as pd and used it to read the data from the excel sheet 'BankChurners'; which we obtained from Kaggle, we used pd.read_csv().

III. ANALYZING THE DATA

The things we wanted to analyze are:

A. What is the shape of the Dataset?

We got a notion of the whole dataset from the following code in Figure 1; we have approximately 10K samples and a total of 23 columns, the most of which are numerical and a few of which are categorical.

```
In [3]: bank.shape
```

```
Out[3]: (10127, 23)
```

Fig 1. Bank.shape

B. What features are we working with?

```
In [5]: bank.columns
Out[5]: Index(['CLIENTNUM', 'Attrition_Flag', 'Customer_Age', 'Gender',
              'Dependent_count', 'Education_Level', 'Marital_Status',
              'Income_Category', 'Card_Category', 'Months_on_book',
              'Total_Relationship_Count', 'Months_Inactive_12_mon',
              'Contacts_Count_12_mon', 'Credit_Limit', 'Total_Revolving_Bal',
              'Avg_Open_To_Buy', 'Total_Amt_Chng_Q4_Q1', 'Total_Trans_Amt',
              'Total_Trans_Ct', 'Total_Ct_Chng_Q4_Q1', 'Avg_Utilization_Ratio',
              'Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_Count_12_mon_Dependent_count_Education_Level_Months_Inactive_12_mon_1',
              'Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_Count_12_mon_Dependent_count_Education_Level_Months_Inactive_12_mon_2'],
              dtype='object')
```

Fig. 2. Bank.columns

By using the code “bank.columns” we notice 3 columns that has nothing to add to our project, so we dropped them (client numbee and the last two columns which are a combination of data) as you can see in the code (IN[6]).

```
bank = bank.drop(['CLIENTNUM',
                  'Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_Count_12_mon_Dependent_count_Education_Level_Months_Inactive_12_mon_1',
                  'Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_Count_12_mon_Dependent_count_Education_Level_Months_Inactive_12_mon_2'], axis = 1)
```

C. What is the type of each feature?

From this code’s output we can know which of the data we have to encode in 3.1, and we can also notice that there are no missing values in the data set (all the features got the same non-null values as the total samples).

```
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                ---
0   Attrition_Flag                        10127 non-null  object
1   Customer_Age                         10127 non-null  int64
2   Gender                              10127 non-null  object
3   Dependent_count                      10127 non-null  int64
4   Education_Level                      10127 non-null  object
5   Marital_Status                      10127 non-null  object
6   Income_Category                     10127 non-null  object
7   Card_Category                       10127 non-null  object
8   Months_on_book                      10127 non-null  int64
9   Total_Relationship_Count            10127 non-null  int64
10  Months_Inactive_12_mon              10127 non-null  int64
11  Contacts_Count_12_mon               10127 non-null  int64
12  Credit_Limit                       10127 non-null  float64
13  Total_Revolving_Bal                 10127 non-null  float64
14  Avg_Open_To_Buy                    10127 non-null  float64
15  Total_Amt_Chng_Q4_Q1               10127 non-null  float64
16  Total_Trans_Amt                    10127 non-null  int64
```

Fig. 3. Bank.info()

IV. VISUALIZING THE DATA

We imported from the seaborn library the countplot function to plot the attrition flag with most features, such as:

Gender, Marital Status, Customer Age, Education Level, Card Category, Dependent count, Months on book, Months_Inactive_12_mon, Months_Inactive_12_mon with the Gender, Total Relationship Count, Contacts_Count_12_mon, Credit_Limit, Avg_Open_To_Buy, Months_Inactive_12_mon, Total_Trans_Amt with the income category.

And by that we could conclude that the most important features due to the figures are:

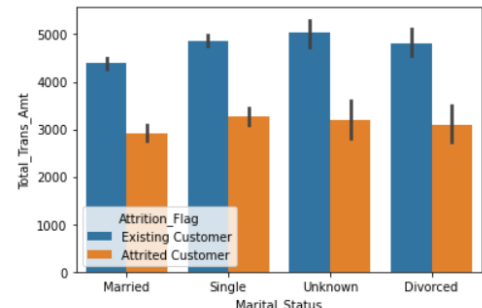
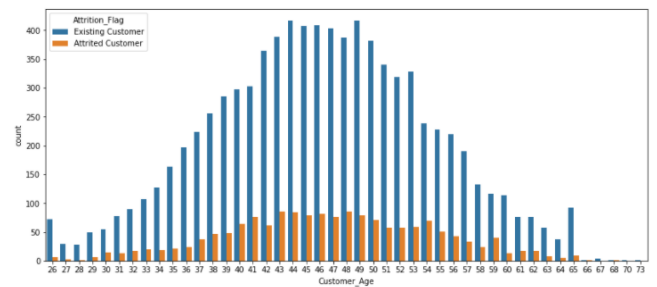


Fig. 5. Visualizing 2

We can see that people from age 26 to 35 and 56 to 65 are less likely to attrit but most likely the ages between are attriting, and single people are more likely to attrit too.

V. METHODOLOGY

A. Encoding the Data

In many Machine-learning or Data Science activities, the data set might contain text or categorical values (basically non-numerical values) and most of the algorithms expect numerical values to achieve results. After knowing which features contains objects from the code in Figure 3, we made another code that shows them in one table.

```
In [31]: obj_bank = bank.select_dtypes(include=['object']).copy()
obj_bank.head()
```

```
Out[31]:
```

	Attrition_Flag	Gender	Education_Level	Marital_Status	Income_Category	Card_Category
0	Existing Customer	M	High School	Married	60K–80K	Blue
1	Existing Customer	F	Graduate	Single	Less than \$40K	Blue
2	Existing Customer	M	Graduate	Married	80K–120K	Blue
3	Existing Customer	F	High School	Unknown	Less than \$40K	Blue
4	Existing Customer	M	Uneducated	Married	60K–80K	Blue

Fig. 6. objects

After that we started applying a code that shows the different outcomes of each feature using this code: `obj_bank["Feature Name"].value_counts()`

Then we used two types of Encoding: Label Encoding and One-Hot Encoding inside the variable `cleanup_nums` then we replaced it inside the bank data.

```
In [37]: bank = bank.replace(cleanup_nums)
bank.head()

Out[37]:
```

	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	Total_Relation
0	1	45	0	3	1	0	2	0	39	
1	1	49	1	5	3	1	0	0	44	
2	1	51	0	3	3	0	3	0	36	
3	1	40	1	4	1	2	0	0	34	
4	1	40	0	3	0	0	2	0	21	

Fig. 7. Replacing the encoded data inside the dataset

As a result, all our data now is numerical values.

B. Splitting the Data

We use the train split for actual training and the test split to measure the model performance and in order to measure how a model performs on new instances we have split the data into 75% train set and 25% test set; from `sklearn.model_selection` library we imported `train_test_split` to split the data which led that the train set has 7595 data, and the test set has 2532 data.

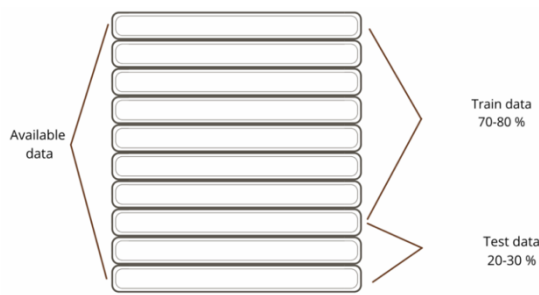


Fig. 8. Data splitting

VI. CREATING THREE MACHINE LEARNING MODELS

A. K-NN Algorithm

We started our modelling with K-Nearest Neighbor Algorithm.

K nearest neighbors is a supervised learning algorithm, it is a simple algorithm for classifying data items based on their closest neighbors. It makes a "educated guess" on what an unclassified point should be classed as based on test results. The KNN contains a few hyperparameter. The most important one is:

n_neighbors: When voting, this is the number of neighbors to utilize. The greater the number, the more precise it becomes (sometimes), but at the cost of speed.

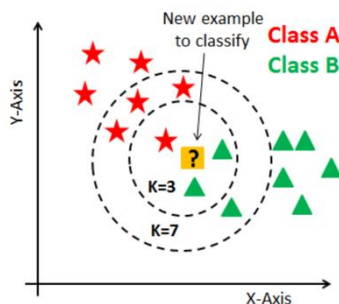


Fig. 9: K-NN Algorithm

We tried 3 different values for the `n_neighbors` and by using the code in fig. 10. the results were arranged to give us the best scores results for our 3 iterations.

```
In [58]: scores = [acc_k_3*100, acc_k_5*100, acc_k_15*100]
Models = pd.DataFrame({
    'Model': ["k=3", "k=5", "k=15"],
    'Score (%)': scores})
Models.sort_values(by='Score (%)', ascending=False)
```

```
Out[58]:
```

Model	Score (%)
1 k=5	88.625592
2 k=15	88.625592
0 k=3	88.309637

Fig. 10: K-NN Algorithm results

B. Random Forest Algorithm

Next, we used another simple but efficient algorithm called decision tree.

Random forest is a supervised learning algorithm, it creates many decision trees and then combines them to provide a more accurate and stable estimate.

Random Forest contains may hyperparameter, but the most important ones are:

1) **n_estimators:** This is the total number of decision trees that will be made. As one might expect, a larger number of trees leads to a stronger model, but at the cost of longer training time.

2) **max_depths:** This is the particular tree's maximum depth. The higher the number, the simpler the trees and therefore the random forest ensemble.

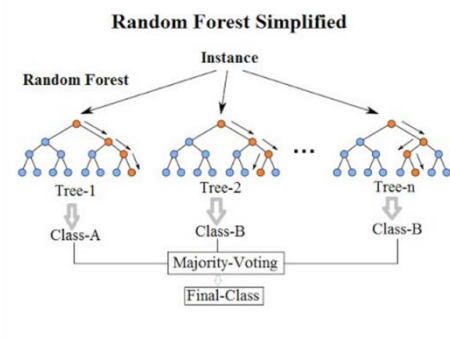


Fig. 11: Random Forest Algorithm

We tried 4 different values for `n_estimators` with same value of `max_depths`, and by using the code in Fig. 12; the results were arranged to give us the best score results for our 4 iterations.

```
In [77]: scores = [acc_n_75*100, acc_n_70*100, acc_n_80*100, acc_n_40*100]
Models = pd.DataFrame({
    'Model': ["n_estimators=75", "n_estimators=70", "n_estimators=80", "n est
    'Score (%)': scores})
Models.sort_values(by='Score (%)', ascending=False)
```

```
Out[77]:
```

Model	Score (%)
0 n_estimators=75	96.287520
1 n_estimators=70	96.090047
2 n_estimators=80	96.090047
3 n_estimators=40	95.892575

Fig 12. Random Forest Algorithm results

C. XGBoost Algorithm

XGBoost stands for "Extreme Gradient Boosting," and it has a tremendous amount of predictive power, making it the ideal choice for event accuracy since it combines both a linear model and a tree learning method, making the process about 10 times quicker than previous gradient booster approaches.

This algorithm is effective, quick, and easy to use right out of the box. It's an advanced version of Gradient Boosting that's also incredibly efficient. The fact that the XGBoost is also known as a regularized boosting approach is one of the most interesting aspects of it. This aids in the reduction of overfit modeling.

One of the important hyperparameter here is:

max_depth: This is used to specify how deep each tree node can go. Higher depth allows for more complicated models, and vice versa.

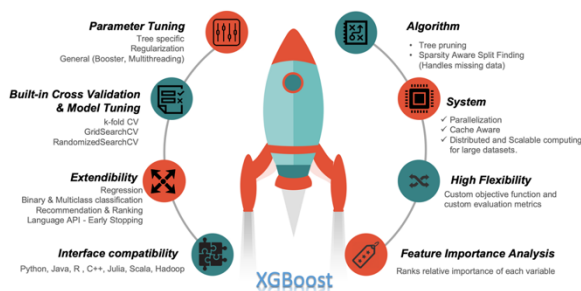


Fig 13. XGBoost Algorithm

We used 70 as the max depth, and it gave us Accuracy as 96.406.

```
In [85]: acc_XGB = accuracy_score(y_test,XGB_predict)
print("Accuracy Score : ",acc_XGB*100)
```

Accuracy Score : 96.40600315955766

Fig 14. XGBoost Algorithm results

D. Comparison between the three models

Our results shows that the best model accuracy was when we used the XGBoost Algorithm which was 96.41% accuracy, as Fig. 15 shows.

```
In [86]: scores = [acc_k_5*100,acc_n_75*100,acc_XGB*100]
Models = pd.DataFrame({
    'Model': ["K Nearest Neighbors","Random Forest","XGBoost"],
    'Score (%)': scores})
Models.sort_values(by='Score (%)', ascending=False)
```

Out[86]:

	Model	Score (%)
2	XGBoost	96.406003
1	Random Forest	90.287520
0	K Nearest Neighbors	88.825592

Fig 15. Results

VII. CONCLUSION

In conclusion, the XGBoost algorithm was the best algorithm for our classification problem because it gave us the best accuracy.

And based on our confusion matrix we can see that it predicted: 2076 True Positive, 37 False Negative, 365 True Negative, and 54 False Positive.

Which led to these results:

```
In [84]: print(classification_report(y_test, XGB_predict))
```

	precision	recall	f1-score	support
0	0.91	0.87	0.89	419
1	0.97	0.98	0.98	2113
accuracy			0.96	2532
macro avg	0.94	0.93	0.93	2532
weighted avg	0.96	0.96	0.96	2532

Fig. 16. Results 2

REFERENCES

- [1] 2021. [Online]. Available: <https://www.expert.ai/blog/machine-learning-definition/>
- [2] "Categorical encoding using Label-Encoding and One-Hot-Encoder", Medium, 2021. [Online]. Available: <https://towardsdatascience.com/categorical-encoding-using-label-encoding-and-one-hot-encoder-911ef77fb5bd>
- [3] C. Moffitt, "Guide to Encoding Categorical Values in Python - Practical Business Python", Pbppython.com, 2021. [Online]. Available: <https://pbpython.com/categorical-encoding.html>
- [4] "Categorical encoding using Label-Encoding and One-Hot-Encoder", Medium, 2021. [Online]. Available: <https://towardsdatascience.com/categorical-encoding-using-label-encoding-and-one-hot-encoder-911ef77fb5bd>
- [5] "A complete guide to the random forest algorithm", Built In, 2021. [Online]. Available: <https://builtin.com/data-science/random-forest-algorithm>
- [6] "Understanding XGBoost Algorithm | What is XGBoost Algorithm?", GreatLearning Blog: Free Resources what Matters to shape your Career!, 2021. [Online]. Available: <https://www.mygreatlearning.com/blog/xgboost-algorithm/>